triangle.py

```
for i in range(1, 6): # Outer loop for the number of rows (1 to 5)
    for j in range(1, i + 1): # Inner loop for printing numbers (1 to i)
        print(j, end='') # Print the number without a newline character
    print() # Move to the next line after each row
```

Variables

Name	Value

Input

Output

triangle2.py

```
for i in range(5, 0, -1):  # Outer loop for the number of rows (5 to 1)
    for j in range(1, i + 1):  # Inner loop for printing numbers (1 to i)
        print(j, end='')  # Print the number without a newline character
    print()  # Move to the next line after each row
```

Variables

Name	Value

Input

Output

- Using Python, write nested the loops to generate the following patterns:

1 22 333 4444 55555	5 4 3 2 1 4 3 2 1 3 2 1 2 1 1	5 4 3 2 1 5 4 3 2 5 4 3 5 4 5	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

- The following function aims to find the largest value in the diagonal of a square matrix. You are given the definition of the function and the description of its inputs and outputs. Complete the function.

```python
def FindLargestDiag(M):
    """

    :param M: The square matrix
    :return: max_pos: the (x,y) coordinate of the largest value in the diagonal
    :return: max_val:  the largest value in the diagonal
    """

    # Initialize variables to store the maximum value and its position
    max_val = -1
    max_pos = (-1,-1)

    # YOUR CODE HERE

    return max_pos, max_val


# Example usage:
M = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

pos, val = FindLargestDiag(M)
print(f"The largest diagonal element is {val} at position {pos}")
```

- An Armstrong number (also known as a narcissistic number or a pluperfect digital invariant) is a number that is equal to the sum of its own digits, each raised to the power of the number of digits. In other words, an n-digit number is considered an Armstrong number if the sum of its individual digits, each raised to the nth power, is equal to the number itself.
  For example, let's take the number 153:
  - The number of digits in 153 is 3.
  - Each digit of 153 raised to the power of 3 is: 1^3 = 1, 5^3 = 125, and 3^3 = 27.
  - The sum of these results is: 1 + 125 + 27 = 153.

  Here are a few more examples of Armstrong numbers:

  0 (Trivially, as 0^1 = 0)
  1 (Trivially, as 1^1 = 1)
  370 (3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370)
  407 (4^3 + 0^3 + 7^3 = 64 + 0 + 343 = 407)
  1634 (1^4 + 6^4 + 3^4 + 4^4 = 1 + 1296 + 81 + 256 = 1634)

  Complete the following program which checks if a number is an Armstrong number of not:

```python
def power(x, y):
    """
    Function to calculate the power of a number
    :param x: first number
    :param y: second number
    :return: x to the power y
    """
    # COMPLETE CODE HERE
def count_digits(num):
    """
    Function to find the number of digits in a number
    :param num: input
    :return count: the number of digits in num
    """
    count = 0
    # COMPLETE CODE HERE
    return count
```

```python
def is_armstrong(num):
    """
    Function to check if a number is an Armstrong number
    :param num: input
    :return: True if Amrstrong, False otherwise
    """
    original_num = num
    num_of_digits = count_digits(num)
    sum_of_powers = 0


    # COMPLETE CODE HERE
    # Write a while loop that computes the sum of powers
    # Example: for num = 153, the sum_of_powers = 1^3 + 5^3 + 3^3 = 153

    if original_num == sum_of_powers:
        return True
    else:
        return False


# Input from the user
num = int(input("Enter a number: "))

# Check if the number is an Armstrong number
if is_armstrong(num):
    print(f"{num} is an Armstrong number.")
else:
    print(f"{num} is not an Armstrong number.")
```